

Session 2.3 : Binary Computations

Session 2.3: Focus

- Subtraction using 2's Complement
 - 2's Complement Notation
 - 2's complement subtraction
- Universal Gates
 - NAND and NOR

Subtraction Using 2' Complement Arithmetic

1's and 2's Complement

- **1's Complement:**

1 0 1 1 0 0 1 0	Binary number
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓	
0 1 0 0 1 1 0 1	1's complement

- **2's Complement:**

10110010	Binary number
01001101	1's complement
+ 1	Add 1
01001110	2's complement

Why these steps?

2's Complement Explained

- 2's complement is a method of **performing subtraction** using **addition operation**
- Assume, we want to perform: **(9 – 5)**
- It is the same as: **(9) + (-5) → (9) + (0 - 5)**
- Let us assume a **4 bit ALU**, for simplicity

In binary, subtracting any number from all 1's is nothing but complementing each bit
i.e., $1111 - 0101 \rightarrow 1010$
(complementing all bits)

- **Let us find out (0 – 5)**

$$\begin{array}{r} 0000 \\ - 0101 \\ \hline \end{array} \rightarrow \begin{array}{r} 1\ 0000 \\ - 0101 \\ \hline \end{array}$$

$$\begin{array}{r} 0000 \\ - 0101 \\ \hline \end{array}$$

$$\begin{array}{r} 0000 \\ - 0101 \\ \hline \end{array}$$

$$\begin{array}{r} 0000 \\ - 0101 \\ \hline \end{array}$$

So, 2's complement of a number (n)
is nothing but (0 – n)

$$1111 + 1 \rightarrow$$

$$\begin{array}{r} 1 \\ + 1111 \\ - 0101 \\ \hline \end{array}$$

Adding 1

Complementing
All bits

$$\boxed{1011}$$

2's complement of 5

It is same as (0 – 5)

2's Complement Notation

- Negative numbers can be represented using 2's complement notation
- Negative of a number is achieved by complementing all the bits and adding a 1 to it
- For **example**, -3 can be represented in 2's complement, by complementing 011-> 100 and then add 1 to it; **101**
 - Using three bits
- Only one representation for zero
- Simpler HW implementation for adder/subtractor

Bits	Unsigned Value	Two's Complement Value
000	0	0
001	1	1
010	2	2
011	3	3
100	4	-4
101	5	-3
110	6	-2
111	7	-1

2's Complement Subtraction

- Subtraction is similar to addition.
- Adding 2's complement of the subtrahend to the minuend and disregarding the carry, if any, achieves subtraction**
 - Example: $17 - 10 = 7$**
- In 2's complement arithmetic, the **answer** is also in **2's complement notation**

Decimal	Twos Complement	
17	00010001	Minuend
– 10	11110101	Subtrahend
	<u> 1</u>	+ Plus 1
	(1)11100010	Carry
	↓ 00000111	Answer
	Discarded	

Quiz 1: Choose the Correct Option

- If a **32 bit** register has a value **0xFFFF FFFF** stored in it, as per 2's complement notation, the equivalent **decimal value** is:

- A. -1
- B. $(2^{32} - 1)$
- C. Both A and B
- D. None of the above

Answer: A

Note: Assume that the value stored is a **signed integer**.

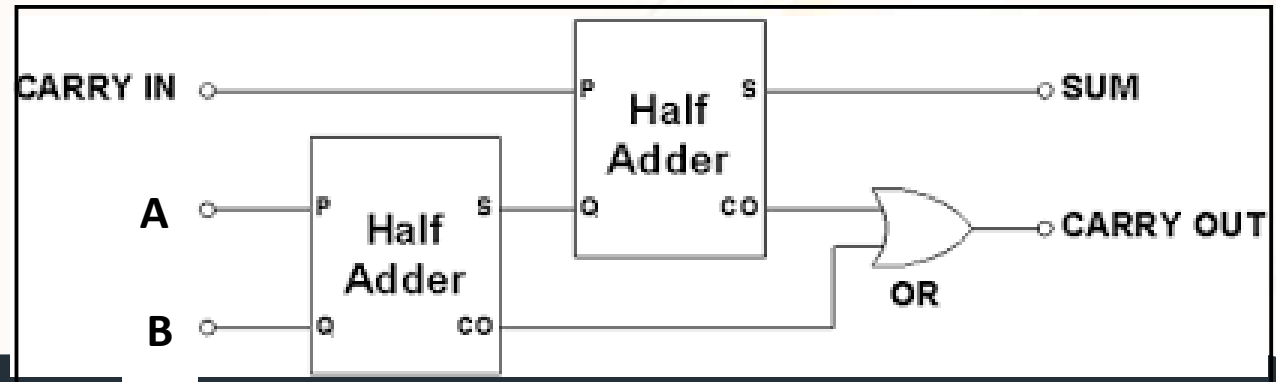
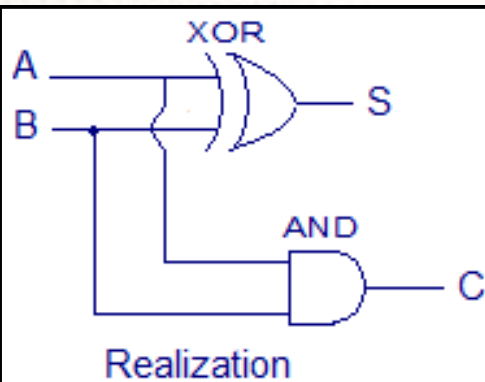
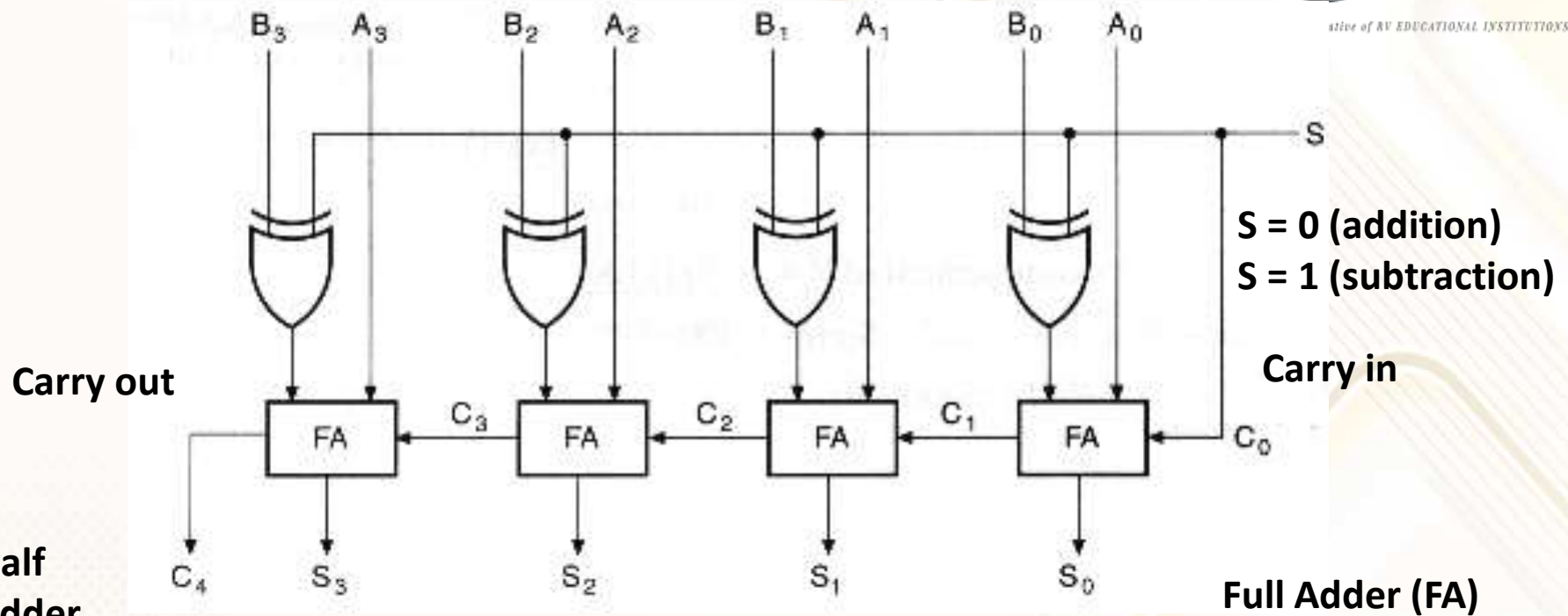
- If a **32 bit** register has a value **0xFFFF FFFF** stored in it, as per 2's complement notation, the equivalent **decimal value** is:

- A. -1
- B. $(2^{32} - 1)$
- C. Both A and B
- D. None of the above

Answer: B

Note: Assume that the value stored is a **unsigned integer**.

4-bit Adder/Subtractor Circuit



Universal Gates

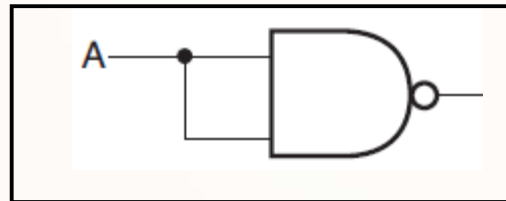
Universal Gates

- **Universal gate** is the one **with** which **all the basic logic gates** can be **built**
 - Basic logic gates are: **AND, OR, NOT**
- Can we build AND, OR, NOT gates **with** only **NAND** gates? - Yes!!!

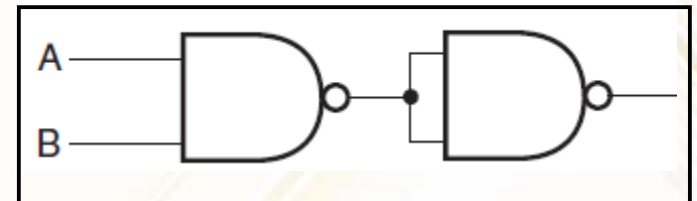
NAND



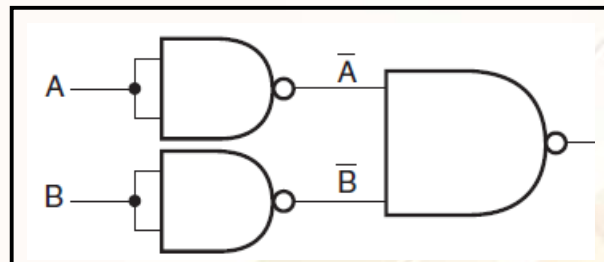
NOT: $Y = \overline{A}$



AND: $A \cdot B$



OR: $A + B$



$Y = A + B$

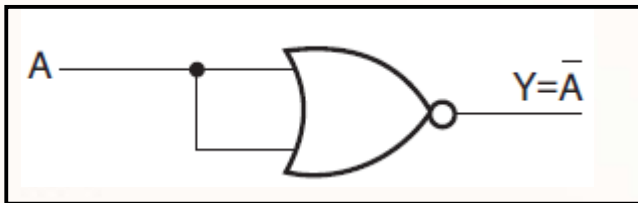
Quiz 1: Is NOR an Universal Gate?

NOR

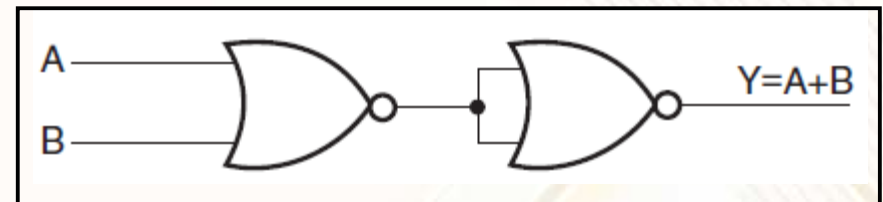


- Yes!!!

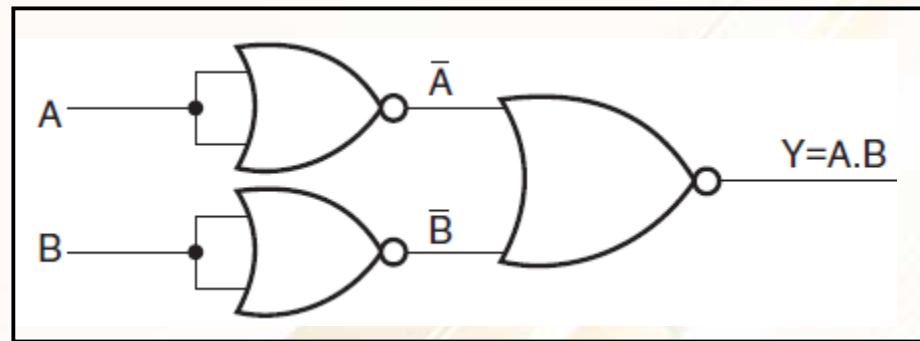
NOT: $Y = \bar{A}$



OR: $A + B$



AND: $A \cdot B$



Binary Left Shift(<<)

➤ Left Shift(<<):

It is a binary operator that takes two numbers, left shifts the bits of the first operand, and the second operand decides the number of places to shift.

In other words, left-shifting an integer “a” with an integer “b” denoted as ‘(a<<b)’

is equivalent to multiplying a with 2^b (2 raised to power b).

Example: Let’s take $a=5$; which is 101 in Binary Form.

Now, if “a is left-shifted by 2” i.e $a=a<<2$ then a will become $a=a*(2^2)$. Thus, $a=5*(2^2)=20$ which can be written as 10100.

Binary Right Shift(<<)

Right Shift(>>):

It is a binary operator that takes two numbers, right shifts the bits of the first operand, and the second operand decides the number of places to shift.

In other words, right-shifting an integer “a” with an integer “b” denoted as ‘**(a>>b)**’ is equivalent to dividing a with 2^b .

Example: let’s take **a=5**; which is **101** in Binary Form. Now, if “a is right-shifted by 2” i.e **a=a>>2** then a will become **a=a/(2^2)**. Thus, **a=a/(2^2)=1** which can be written as **01**.

Session 2.3 Summary

- Subtraction using 2's Complement
 - 2's Complement Notation
 - 2's complement subtraction
- Universal Gates
 - NAND and NOR

Session 2.4 : Theorems in Boolean Algebra

Session 2.4: Focus

- Introduction to Boolean Algebra
- Ordinary Algebra Vs Boolean Algebra
- Boolean Expressions
 - Variables, literals and terms
 - Complement and Dual
- DeMorgan's Theorems

Introduction to Boolean Algebra

George Boole



- **George Boole** (2 November 1815 – 8 December 1864)
- He was an English mathematician, educator, philosopher and logician.
- He worked in the fields of differential equations and algebraic logic, and is best known as the author of **The Laws of Thought** (1854) which contains **Boolean algebra**.
- Boolean logic is credited with laying the foundations for the information age.

Boolean Algebra

- Founded by **George Boole**, in **1854**.
- Boolean algebra is **mathematics of logic**, a systematic way of expressing and analyzing the operations of **logic circuits**
- It is one of the **most basic tools** available to **the logic designer**
 - It can be effectively **used** for **simplification** of **complex logic expressions**
- Now, let us have a closer look at the different **postulates** and **theorems** of **Boolean algebra**
- Their applications in **minimizing Boolean expressions**

Introduction to Boolean Algebra

- **Boolean algebra**, quite interestingly, is **simpler** than **ordinary algebra**.
- It is also composed of a set of **symbols** and a set of **rules to manipulate** these symbols

Ordinary Algebra Vs Boolean Algebra

Ordinary Algebra Vs Boolean Algebra

Ordinary Algebra	Boolean Algebra
Letter symbols can take on any number of values including infinity	Letter symbols can take on either of two values , that is, 0 and 1
Values assigned to a variable have a numerical significance	Values assigned to a variable have a logical significance
' and + are respectively the signs of multiplication and addition	' means an AND operation and + means an OR operation
A+B is read as A plus B	A+B is read as A OR B Basic logic operations are AND , OR and NOT
	Captures both logic operations and set operations such as intersection , union and complement

Boolean Expressions

Variables, Literals and Terms

- **Variables** are different **symbols** in a **Boolean expression**
- **They** may **take** on the **values '0' or '1'**
- **Complement** of a variable is **not** considered as a **separate variable**
- **Each occurrence** of a **variable** or **its complement** is called a **literal**
- A **term** is the **expression** formed by **literals** and **operations** at **one level**.
- Each **term** **requires** a **gate** and **each variable** within the **term** designates an **input** to the **gate**

$$\overline{A} + A.B$$

Quiz:

Variables : 2
Literals : 3
Terms : 2

Quiz 1: Variables, Literals and Terms

- How many **Variables**, **literals** and **Terms** are there in these **two Boolean expressions**?
- How many **gates** are needed to construct them?

Boolean Expressions	Variables	Literals	Terms	Gates
$F_2 = x'y'z + x'yz + xy'$	3	8	3	NOT : 2 AND : 3 OR : 1
$F_2 = xy' + x'z$	3	4	2	NOT : 2 AND : 2 OR : 1

Note: Assume both 2-input and 3-input gates are available to construct these expressions, without minimization.

Equivalent and Complement of Boolean Expressions

Equivalent and Complement of Boolean Expressions

- **Two** given **Boolean expressions** are said to be **equivalent**
 - If **one** of them **equals '1'** only when the **other equals '1'** and also **one equals '0'** only when the **other equals '0'**
- They are said to be the **complement** of each other
 - If **one** expression **equals '1'** only when the **other equals '0'**, and **vice versa**
- The **complement** of a given **Boolean expression** is **obtained** by
 - **Complementing each literal,**
 - **Changing** all **'.'** to **'+'** and all **'+'** to **'.'**, all **0s** to **1s** and all **1s** to **0s**.

Quiz 1: Find Complements

- Find the **complement** of below **Boolean expressions**:

Boolean Expression	Complement
$\overline{A}.B + A.\overline{B}$	$(A + \overline{B}).(\overline{A} + B)$
$(A + B).(\overline{A} + \overline{B})$	$\overline{A}.\overline{B} + A.B$
$[(A.\overline{B} + \overline{C}).D + \overline{E}].F$	$[(\overline{A} + B).C + \overline{D}].E + \overline{F}$

Dual of Boolean Expressions

Dual of Boolean Expressions

The **dual** of a Boolean expression is obtained by **replacing**

- All **'.'** operations with **'+'** operations
- All **'+'** operations with **'.'** operations,
- All **0s** with **1s** and all **1s** with **0s** and
- Leaving all **literals unchanged**

Quiz 2: Find Dual

- Find the **dual** of below **Boolean expressions**:

Boolean Expression	Dual
$\overline{A}.B + A.\overline{B}$	$(\overline{A} + B).(A + \overline{B})$
$(A + B).(\overline{A} + \overline{B})$	$A.B + \overline{A}.\overline{B}$
$A.\overline{B} + B.\overline{C} + C.\overline{D}$	$(A + \overline{B}).(B + \overline{C}).(C + \overline{D})$

Quiz 3: Find Dual and Complement

- Find the **dual** and **Complement** of below **Boolean expression**:

Boolean Expression	$[(A.\overline{B} + \overline{C}).D + \overline{E}].F$
Dual	$[(A + \overline{B}).\overline{C} + D].\overline{E} + F$
Complement	$[(\overline{A} + B).C + \overline{D}].E + \overline{F}$

What is Dual of a Boolean Expression?

The principle of **duality** pronounces that given an **expression** which is always **valid** in boolean algebra, the **dual expression** is **also** always **valid**

- For example:
 - $A (B + C) = A . B + A . C$
 - It's **dual** is also **valid**
 - $A + (B . C) = (A + B) . (A + C)$

Laws of Boolean Algebra

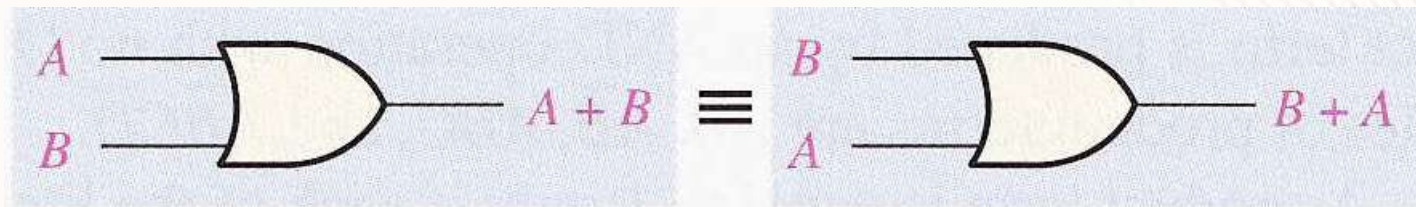
Laws of Boolean Algebra

- The basic **laws** of **Boolean algebra** are:
 - **Commutative laws** for **addition** and **multiplication**
 - **Associative laws** for **addition** and **multiplication**
 - **Distributive law**
- Each law is illustrated with two or three variables, but the number of variables is not limited to this

Commutative Laws

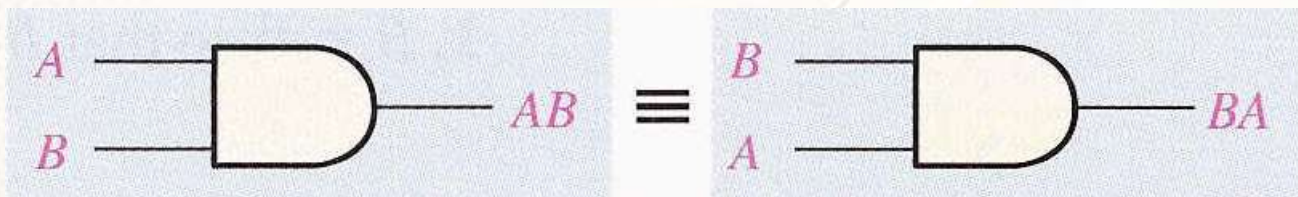
- **Commutative law of addition (OR) for two variables is written as:**

- **$A + B = B + A$**



- **Commutative law of multiplication (AND) for two variables is written as:**

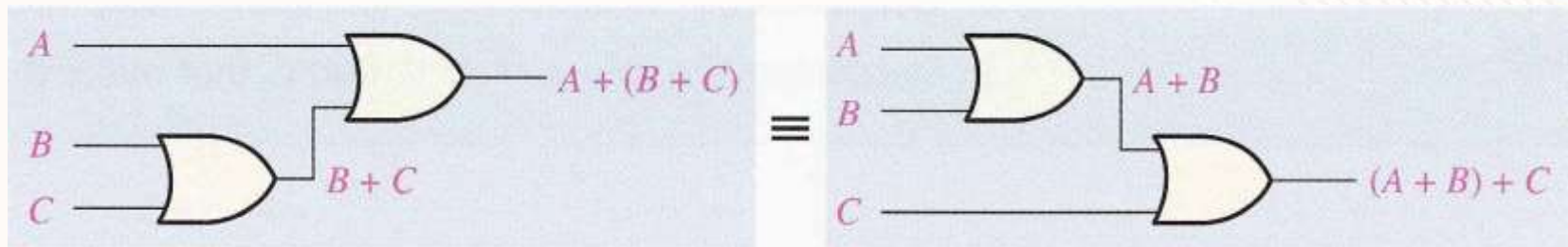
- **$A \cdot B = B \cdot A$: it can also be written as $AB = BA$**



Associative Laws

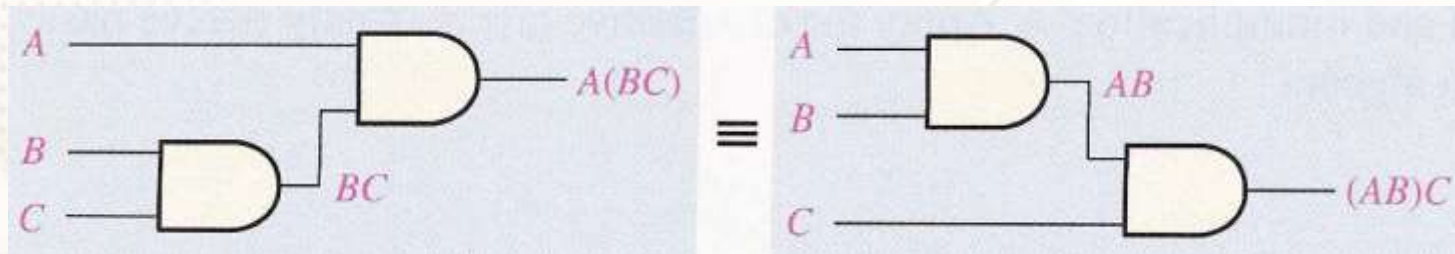
- **Associative law of addition (OR) for three variables** is written as:

- $A + (B + C) = (A + B) + C$



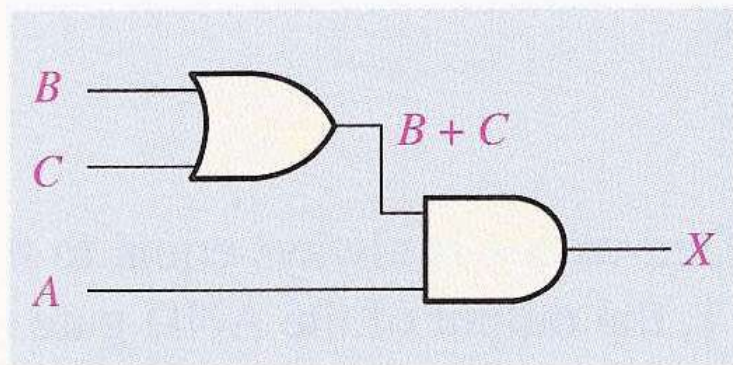
- **Associative law of multiplication (AND) for three variables** is written as:

- $A \cdot (B \cdot C) = (A \cdot B) \cdot C$



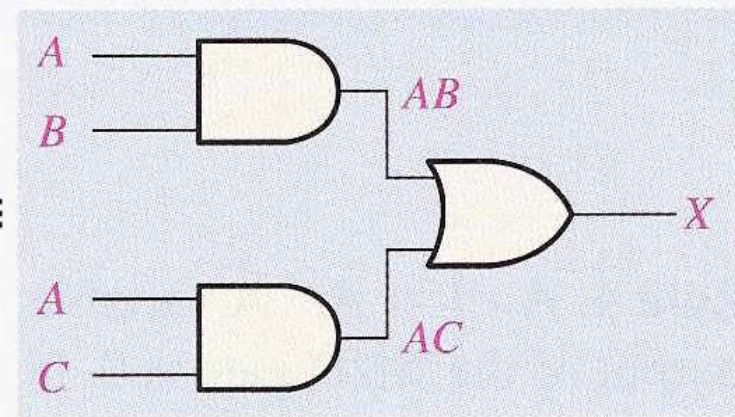
Distributive Law

- **Distributive law for three variables** can be written as:
 - $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$



$$X = A(B + C)$$

$\equiv$



$$X = AB + AC$$

Note that **only . over + is distributive**. But, **+ over . is not distributive**

$$A + (B \cdot C) \neq (A + B) \cdot (A + C)$$

Rules of Boolean Algebra

Rules of Boolean Algebra

- **A, B or C** can represent a single variable or a combination of variables

$$1. A + 0 = A$$

$$2. A + 1 = 1$$

$$3. A \cdot 0 = 0$$

$$4. A \cdot 1 = A$$

$$5. A + A = A$$

$$6. A + \bar{A} = 1$$

$$7. A \cdot A = A$$

$$8. A \cdot \bar{A} = 0$$

$$9. \bar{\bar{A}} = A$$

$$10. A + AB = A$$

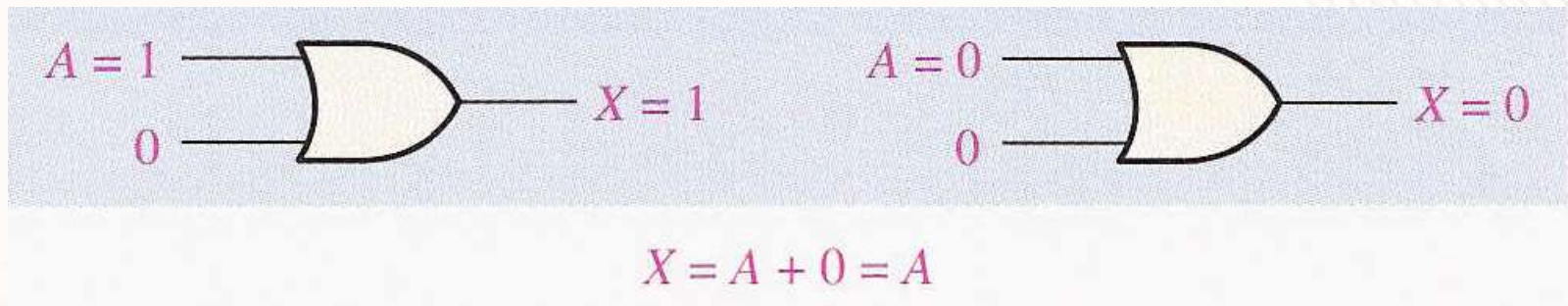
$$11. A + \bar{A}B = A + B$$

$$12. (A + B)(A + C) = A + BC$$

- These **12 rules** are useful in **simplifying** Boolean expressions

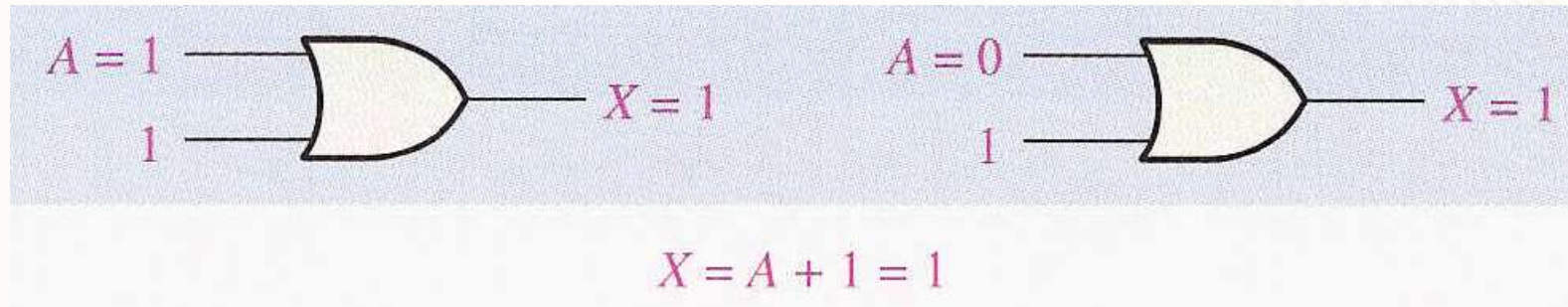
Rule 1

- $A + 0 = A$



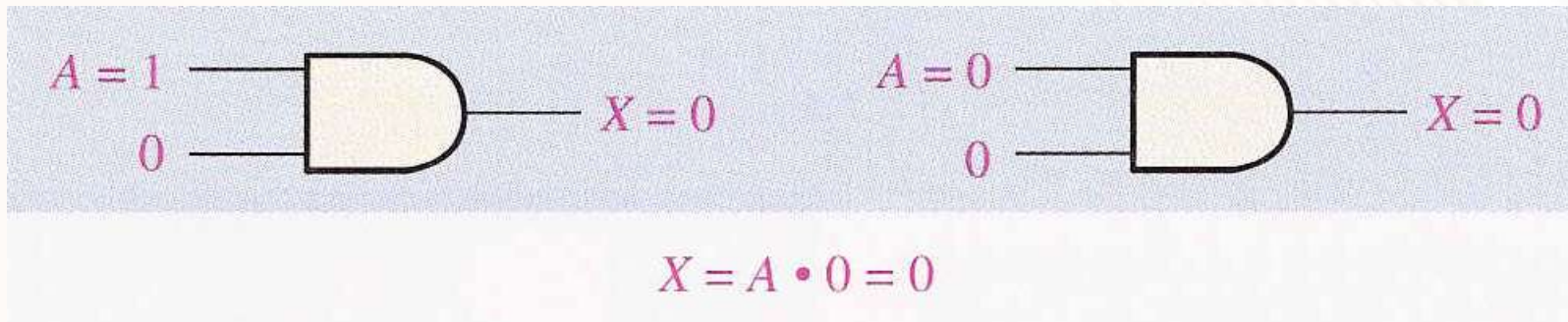
Rule 2

- **$A + 1 = 1$**



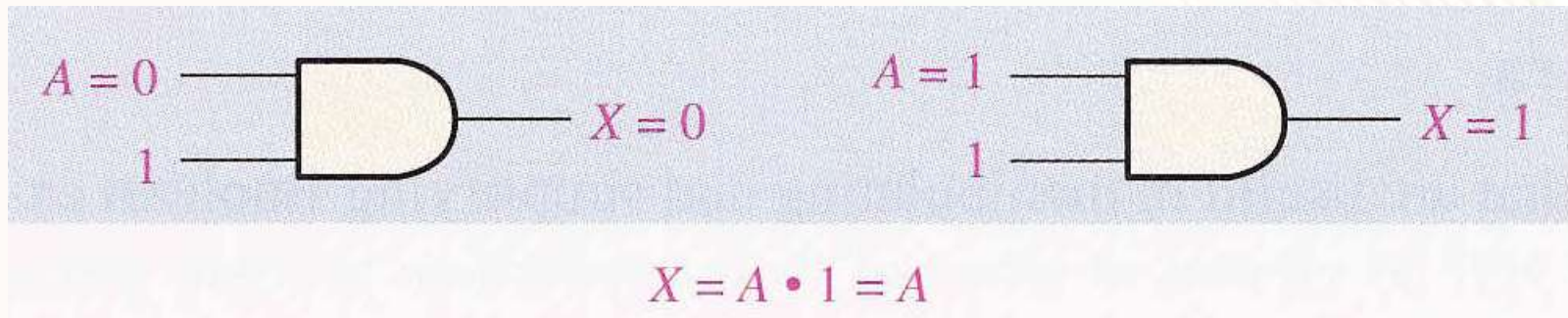
Rule 3

- **$A \cdot 0 = 0$**



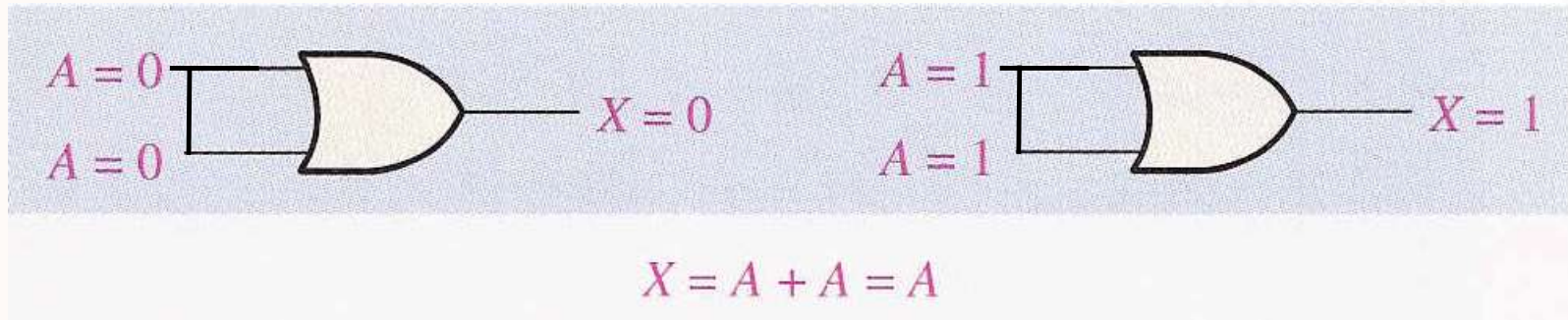
Rule 4

- $A \cdot 1 = A$



Rule 5

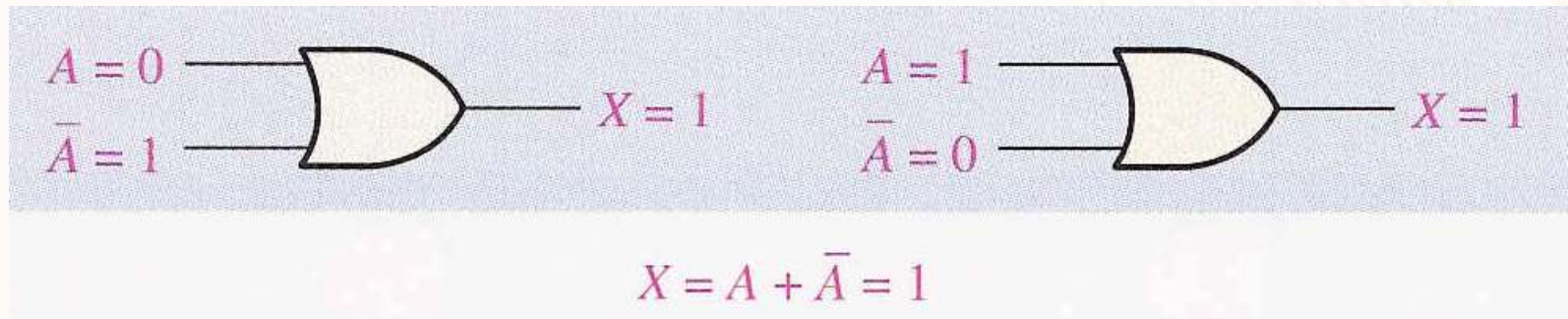
- **$A + A = A$ (Idempotent or Identity Law)**



- Valid for any number of the same input:
- **$A + A + A + + + A = A$**

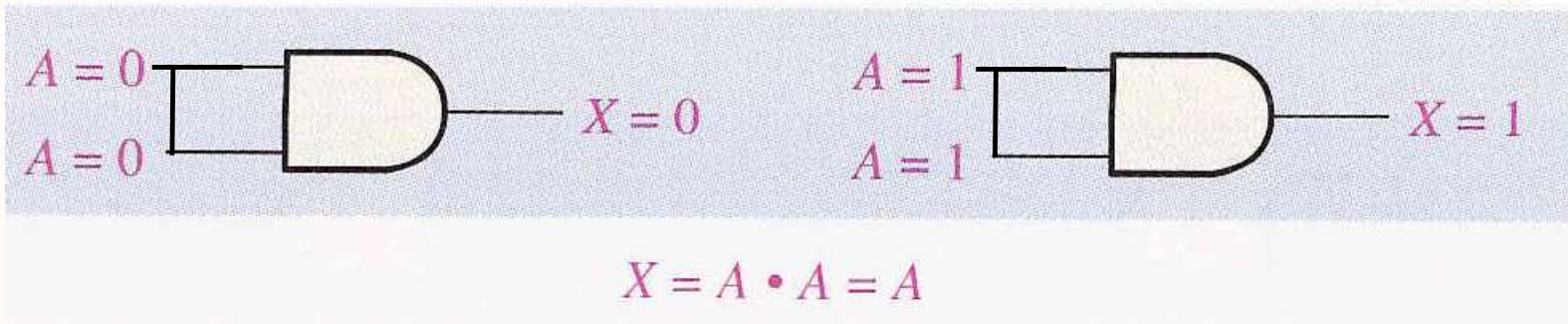
Rule 6

- $A + \bar{A} = 1$



Rule 7

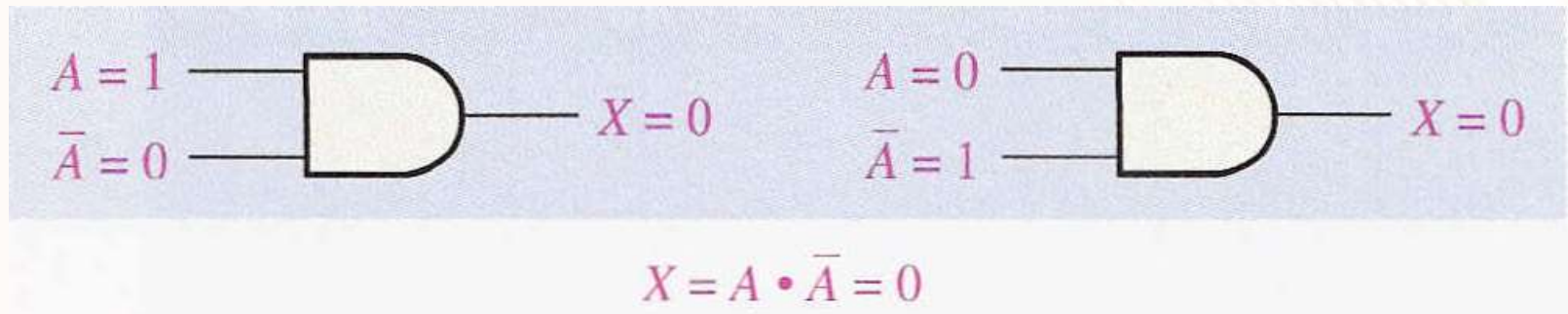
- $A \cdot A = A$ (Idempotent or Identity Law)



- Valid for any number of the same input:
- $A \cdot A \cdot A \dots A = A$

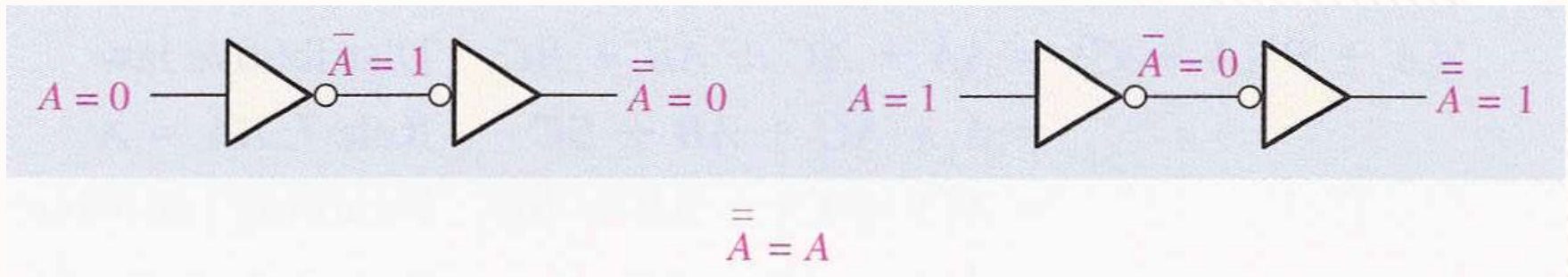
Rule 8

- $A \cdot \bar{A} = 0$



Rule 9

- $\overline{\overline{A}} = A$ (Involution law)

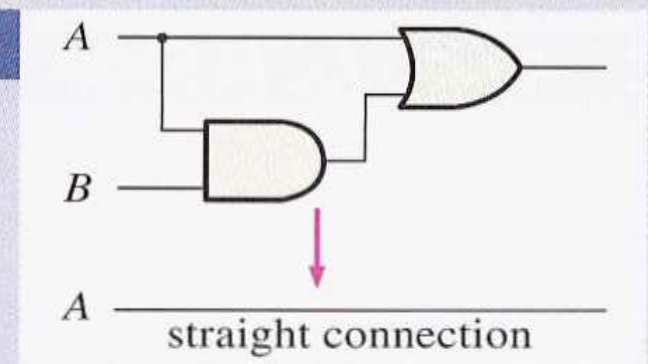


Rule 10

- $A + AB = A$ (Absorption or Redundancy Law)
- $A + AB = A(1 + B)$: Factoring (Distributive law)
- $= A \cdot 1$: By Rule 2, $(1 + B) = 1$
- $= A$: By Rule 4, $A \cdot 1 = A$
- $A + AB = A$

A	B	AB	A + AB
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

↑ equal ↑



Rule 11

- $A + \overline{A}B = A + B$

$$\begin{aligned}
 A + \overline{A}B &= (A + AB) + \overline{A}B \\
 &= (AA + AB) + \overline{A}B \\
 &= AA + AB + A\overline{A} + \overline{A}B \\
 &= (A + \overline{A})(A + B) \\
 &= 1 \cdot (A + B) \\
 &= A + B
 \end{aligned}$$

Rule 10: $A = A + AB$

Rule 7: $A = AA$

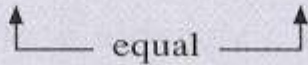
Rule 8: adding $A\overline{A} = 0$

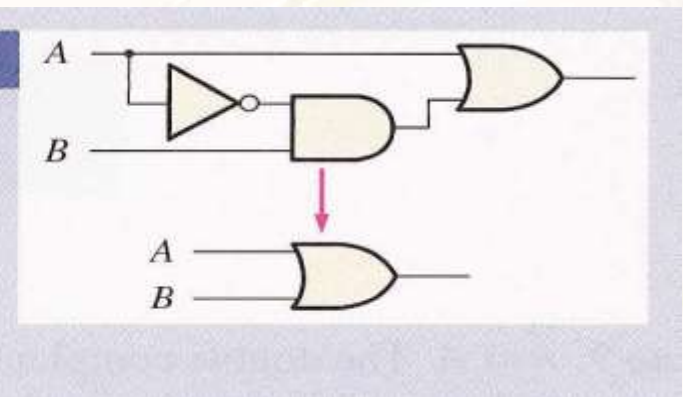
Factoring

Rule 6: $A + \overline{A} = 1$

Rule 4: drop the 1

A	B	$\overline{A}B$	$A + \overline{A}B$	$A + B$
0	0	0	0	0
0	1	1	1	1
1	0	0	1	1
1	1	0	1	1





Rule 12

- **$(A + B) \cdot (A + C) = A + BC$**

$$(A + B)(A + C) = AA + AC + AB + BC \quad \text{Distributive law}$$

$$= A + AC + AB + BC \quad \text{Rule 7: } AA = A$$

$$= A(1 + C) + AB + BC \quad \text{Factoring (distributive law)}$$

$$= A \cdot 1 + AB + BC \quad \text{Rule 2: } 1 + C = 1$$

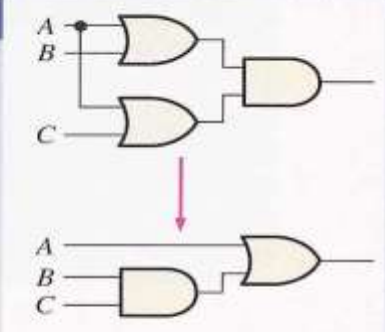
$$= A(1 + B) + BC \quad \text{Factoring (distributive law)}$$

$$= A \cdot 1 + BC \quad \text{Rule 2: } 1 + B = 1$$

$$= A + BC \quad \text{Rule 4: } A \cdot 1 = A$$

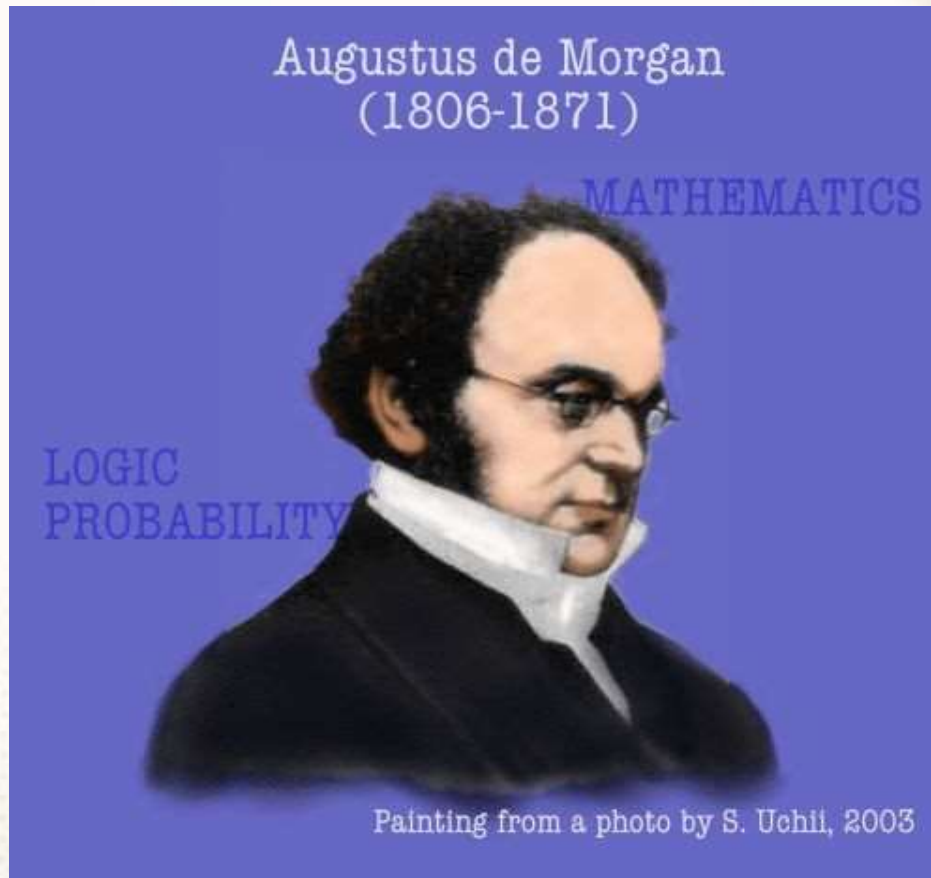
A	B	C	A + B	A + C	(A + B)(A + C)	BC	A + BC
0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	1	1	1	1
1	0	0	1	1	1	0	1
1	0	1	1	1	1	0	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

↑ equal ↑



DeMorgon's Theorems

Augustus De Morgan



Augustus De Morgan,
(born June 27, 1806, Madura, India)
English mathematician and logician
whose major **contributions** to the
study of logic include the formulation
of **De Morgan's laws**

DeMorgan, a mathematician who knew Boole,
proposed two important theorems of Boolean Algebra

DeMorgan's Theorem 1

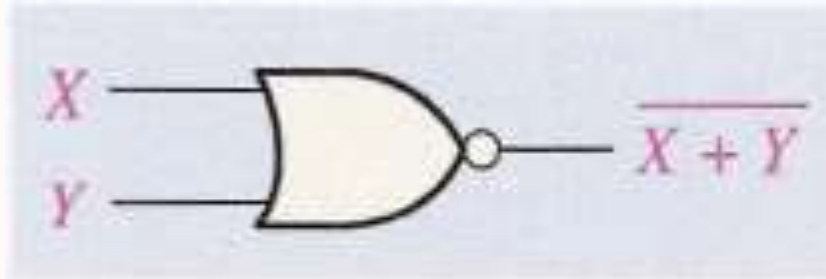
- **Theorem 1:**
- The **complement** of a **sum of variables** is **equal** to the **product of the complements** of the variables

$$\overline{X + Y} = \overline{X} \overline{Y}$$

- It can also be stated as:
- The **complement** of two or more **ORed variables** is **equivalent** to the **AND** of the **complements** of the **individual variables**
- Valid for any number of variables.

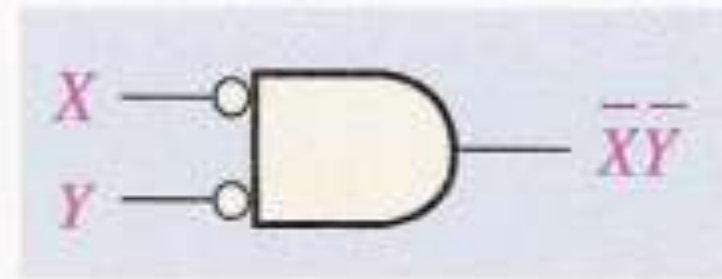
$$\overline{[X_1 + X_2 + X_3 + \dots + X_n]} = \overline{X_1} \cdot \overline{X_2} \cdot \overline{X_3} \cdot \dots \cdot \overline{X_n}$$

DeMorgan's Theorem 1



NOR

$\equiv$



Negative-AND

Inputs		Output	
X	Y	$\overline{X + Y}$	$\overline{\overline{X} \overline{Y}}$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

Session 2.4: Summary

- Introduction to Boolean Algebra
- Ordinary Algebra Vs Boolean Algebra
- Boolean Expressions
 - Variables, literals and terms
 - Complement and Dual
- DeMorgan's Theorems